DynamicSLM: Capability-Modular, Self-Improving Small Language Models for AIKernel

A Model ABI and Runtime Architecture for Dynamic Capability Loading, Differential Distillation, and Heterogeneous Execution

Author: Takuya Sogawa
Affiliation: AIKernel Project
ORCID: https://orcid.org/0009-0009-7499-2595
DOI: https://doi.org/10.5281/zenodo.20550614
Version: v0.1.0
Date: 2026-06-05
Status: Technical Note / Architecture Draft

> The English manuscript is the canonical version. The Japanese manuscript is included as a companion translation.

Abstract

DynamicSLM is a capability-modular Small Language Model (SLM) architecture in which an AIKernel runtime treats model abilities as loadable, governable runtime modules rather than as inseparable regions of a static monolithic model. The central problem addressed by this paper is that current LLMs and SLMs are typically deployed as large indivisible artifacts: once trained, a model behaves as a single function, making it difficult to dynamically add, remove, revoke, replace, schedule, or specialize one capability at runtime.

This paper proposes **DynamicSLM**, a self-improving SLM architecture for AIKernel. DynamicSLM defines each model artifact through an AIKernel Model ABI consisting of a Semantic Profile, Capability Graph, Execution Profile, Lineage, and Payload. These ABI components expose what the model is intended to do, what resources it requires, where it came from, which capability payloads are materialized, and how execution should be governed before the runtime loads any weights.

The main architectural novelty is the separation of model capability from monolithic weight deployment. AIKernel resolves the minimum capability subgraph required for a task, dynamically loads only the corresponding payloads, records execution through ReplayLog-backed semantic traces, and uses differential distillation to convert verified teacher-model trajectories into targeted capability modules. In this way, a local SLM can progressively acquire workload-specific abilities without retraining or redeploying the entire model.

This work is conceptual and architectural in nature; it does not present empirical benchmarks. It specifies the operating-system-level model ABI, capability modularization strategy, distillation lifecycle, and runtime scheduling discipline required to make SLMs behave as governable, hot-swappable, and progressively improving semantic processes inside AIKernel.

Keywords

AIKernel; DynamicSLM; Small Language Models; Capability Graph; Model ABI; Semantic Profile; Execution Profile; Differential Distillation; ReplayLog; Dynamic Capability Loading; Heterogeneous Compute; Semantic Context OS; Self-Improving AI Systems.

1 Introduction

Current LLMs and SLMs are usually designed as static monoliths. Once trained, a model is deployed as a single large parameterized function. Although adapters, routing layers, and tool-use mechanisms

can modify the behavior of such systems, the model itself is still usually managed as a large indivisible artifact. At runtime, it is difficult to load only a specific capability, replace a single ability, revoke a narrow behavioral region, or update a single capability without affecting the rest of the model.

This paper uses an operating-system analogy in a precise architectural sense. The analogy does not mean that a model literally becomes an OS process with all properties of a conventional process. Rather, it means that model capabilities should be declared, admitted, scheduled, isolated, swapped, audited, and revoked by an external runtime boundary in the same way that an operating system governs executable components, dynamic libraries, and resource-constrained processes.

This static model-deployment pattern is increasingly misaligned with the requirements of edge inference, memory-constrained deployment, personalized assistants, and governed AI runtimes. In software engineering, operating systems evolved away from single monolithic binaries toward processes, dynamically linked libraries, memory paging, access control, and resource scheduling. By contrast, modern model deployment still often resembles a pre-dynamic-linking software environment: a single artifact is loaded, and all of its capabilities become available whether or not the current task requires them.

AIKernel approaches this problem from an operating-system perspective. Rather than treating the model as the ultimate execution authority, AIKernel treats inference as a process governed by semantic context, capability contracts, resource profiles, and deterministic execution traces. In this view, a model should not merely be a black-box tensor payload. It should be a governed runtime module whose abilities can be declared, verified, scheduled, isolated, and improved under kernel control.

This paper proposes **DynamicSLM**, a capability-modular and self-improving model architecture fully integrated with AIKernel. DynamicSLM defines a model as a collection of semantic profiles and weight payloads conforming to the **AIKernel Model ABI**. The architecture separates model capability from static weight deployment and exposes capability-level metadata to the kernel before execution.

The terminology used in this note is aligned with the AIKernel Trajectory Governance line [1] and the Phase-2 Semantic Compilation Architecture [2]. DynamicSLM extends these foundations by treating model capability, lineage, execution profile, and payload selection as ABI-level runtime objects.

The contributions of this technical note are threefold.

1. It defines an AIKernel Model ABI that represents a model through five structured elements: Semantic Profile, Capability Graph, Execution Profile, Lineage, and Payload.
2. It introduces a capability-level dynamic loading architecture in which AIKernel resolves the minimum capability subgraph required for a task and loads only the necessary payloads.
3. It proposes a differential distillation loop in which capability gaps are detected during runtime, delegated to teacher models, stored as verified ReplayLog traces, and distilled into targeted capability modules for progressive local improvement.

The goal is not to replace all monolithic models. Some tasks may still benefit from a large coherent model. The goal is to define the runtime architecture needed for SLMs to behave more like operating-system-managed processes: modular, revocable, schedulable, hot-swappable, traceable, and capable of targeted self-improvement.

2 Related Work

DynamicSLM builds on the AIKernel Trajectory Governance foundation, which defines AIKernel as a formal governance framework for a Semantic Context OS and maps stochastic inference into a deterministic safety boundary [1]. It also connects to the Phase-2 Semantic Compilation Architecture, where semantic compilation provides the bridge between abstract intent, structured intermediate representation, and governed execution [2]. Within that lineage, DynamicSLM contributes a model-artifact layer: Model ABI, Capability Graph, Differential Distillation, and heterogeneous scheduling.

Research on modular models has explored techniques such as sparse routing, mixtures of experts, adapters, and low-rank adaptation. Representative examples and families include Sparsely-Gated Mixture-of-Experts, Mixtral, DeepSeek MoE, Phi-3.5 MoE-style modular deployments, LoRA, and QLoRA.

These methods improve parameter efficiency and can enable selective activation of model subcomponents. However, they are generally optimized inside a model architecture. They do not by themselves define an operating-system-level contract by which an external runtime can inspect, verify, admit, revoke, schedule, and audit a capability as a first-class execution object.

AIKernel requires a different boundary. The kernel does not merely ask the model to route probabilistically among internal experts. It requires an explicit contract that states which capability is being loaded, what Semantic Profile it claims, what resources it consumes, how it was derived, and which Payload should be materialized for a given task. DynamicSLM therefore treats model modularity as a runtime governance problem rather than only a parameter-efficiency problem.

Tool-use and function-calling approaches allow LLMs to invoke external capabilities. Toolformer, OpenAI Function Calling, LangGraph, and LangChain-style agents represent important examples of this direction. These techniques are useful for extending model behavior, but they often leave the model artifact itself unchanged. The tool is external, while the model remains a monolithic planner or controller. DynamicSLM moves part of the capability structure into the model artifact itself while still keeping it governed by AIKernel's external capability registry.

Research on recursive self-improvement, continual adaptation, preference optimization, and trajectory-based improvement explores pipelines in which models evaluate their outputs and update themselves over time. Examples include Self-Refine, Reflexion, DPO, ORPO, trajectory distillation in reasoning-focused models such as DeepSeek-R1, and broader discussions of recursive self-improvement in frontier-model research. However, such pipelines often operate through prompt engineering, full-model fine-tuning, preference optimization over broad behavior, or broad adapter updates. DynamicSLM narrows the unit of improvement to a declared Capability node. The system detects capability gaps, delegates them to a stronger teacher model when needed, and distills the resulting verified traces back into the smallest relevant capability module.

Recent AI operating system, inference serving, and agent runtime systems include Microsoft Semantic Kernel, NVIDIA NIM, Triton Inference Server, Modal, Ray Serve, and related orchestration layers. These systems provide important runtime and deployment mechanisms, but they do not directly define a model artifact as a capability-governed shared-library-like unit with Semantic Profile, Execution Profile, Lineage, and Payload under a deterministic kernel boundary.

DynamicSLM differs by applying AIKernel's deterministic governance and semantic-context separation directly to model weight management. The SLM is reconstructed as a shared library-like component governed by a Model ABI, a Capability Graph, a Lineage chain, and a resource-aware runtime loader.

3 DynamicSLM Architecture

The core of DynamicSLM is the **AIKernel Model ABI**. This ABI is a standardized interface that allows AIKernel to reason about a model before loading it. In conventional deployment, a model is primarily a tensor payload plus tokenizer and configuration files. In DynamicSLM, the payload is only one component of a broader governed artifact.

A DynamicSLM artifact consists of five elements.

- **Semantic Profile:** The semantic contract of the model or capability module, including input assumptions, output schema guarantees, bounded semantic scope, and expected task domains.
- **Capability Graph:** A directed acyclic graph representing the capabilities provided by the model and the dependencies among them.
- **Execution Profile:** A declaration of runtime resource requirements, including memory footprint, expected latency, compute shape, accelerator suitability, and scheduling constraints.
- **Lineage:** A provenance record describing how the artifact was produced, which parent model or teacher model was used, which ReplayLogs were used for distillation, and which cryptographic hashes identify the artifact.
- **Payload:** The actual weight tensors, adapters, quantized blocks, tokenizer fragments, or executable model segments required to realize the declared capability.

These five components collectively define the ABI boundary between AIKernel and DynamicSLM artifacts: the Semantic Profile describes the contract, the Capability Graph describes dependency structure, the Execution Profile declares resource shape, the Lineage proves provenance, and the Payload carries the materialized model component.

This structure allows AIKernel to fail closed before model loading. If a model claims a capability but lacks a valid lineage, incompatible execution profile, malformed semantic contract, or unverifiable payload hash, the loader can reject it without ever materializing the weights in memory.

3.1 Capability Graph

The Capability Graph represents model capability as a dependency-aware DAG. Each node corresponds to a semantic ability, and each edge expresses a dependency required for correct execution. For example, a `SQL generation` capability may depend on `schema understanding`, `logical reasoning`, and `structured output formation`.

This graph allows AIKernel to resolve a task intent into a minimal capability subgraph. Instead of loading an entire monolithic model for every task, the kernel identifies the capabilities required by the current intent and selects only the associated payloads. This makes the model runtime closer to dynamic linking than to static whole-binary execution.

The graph also supports governance. A policy may allow `read-only summarization` while denying `system command generation`. Because capabilities are explicit nodes, the Policy Decision Point can reject the loading of a dangerous capability before generation begins.

3.2 Semantic Profile

The Semantic Profile defines the contract that a capability module claims to satisfy. It may include the following fields:

- accepted input domains;
- output schema or structured response guarantees;
- semantic bounds or task ellipsoids;
- failure modes and required fallback behavior;
- dependency assumptions over other capabilities;
- compatibility with AIKernel context and ReplayLog formats.

The Semantic Profile is not a prompt. It is a declarative contract exposed to the runtime. AIKernel uses it to determine whether a capability is admissible for a given task and whether its outputs can be consumed by downstream pipeline steps.

3.3 Execution Profile

The Execution Profile declares the operational cost of a capability. It includes resource information such as expected VRAM usage, CPU or NPU suitability, quantization format, tensor shape, expected latency, and prefetch requirements. The runtime scheduler uses this profile to decide where and when to load a payload.

This profile is especially important for edge devices. A small device may not be able to load a full 7B-class model, but it may be able to load a 1.0B to 1.5B capability module and a small adapter. DynamicSLM is designed to make that choice visible to the kernel before the model is loaded.

3.4 Lineage and Payload Verification

Lineage provides provenance and integrity. Each capability payload is associated with a hash chain that records the parent model, teacher model, distillation corpus, ReplayLog set, training configuration, and output artifact hash. The loader verifies this lineage before admitting the payload.

This prevents unauthorized model replacement and supports reproducibility. If a capability behaves unexpectedly, AIKernel can trace which teacher, data, and distillation process produced it.

4 Dynamic Distillation Pipeline

DynamicSLM is not intended to be a static artifact. It is designed to evolve under AIKernel governance. The mechanism that drives this evolution is the **Differential Distillation Pipeline**.

When AIKernel receives a task, it resolves the required capability subgraph. If the current local SLM lacks a capability or fails to satisfy the required semantic profile, the system may delegate the task to a stronger teacher model. This fallback is not treated as an opaque external call. The teacher's reasoning trajectory, output, validation result, and execution context are recorded as ReplayLog-backed training material.

Once enough verified examples accumulate for a capability gap, AIKernel may trigger differential distillation in the background. The system does not retrain the entire model. Instead, it targets a specific Capability Graph node. For example, a repeated failure in domain-specific JSON structuring may produce a small adapter or payload update only for that capability. The concrete update mechanism is not limited to LoRA-style adapters; it may also be expressed as QLoRA deltas, conventional adapters, block-level updates, segment-level payload replacement, quantized tensor patches, or another capability-scoped artifact type admitted by the Model ABI.

The resulting capability module is packaged as a new payload with its own Semantic Profile, Execution Profile, and Lineage. AIKernel registers the new artifact only after integrity checks, validation, and policy admission. The new module then becomes available for future routing.

This creates a **Dynamic Self-Improvement** loop.

1. A task requires a capability.
2. The local SLM capability is missing or insufficient.
3. AIKernel delegates to a teacher model under governance.
4. The successful trajectory and output are stored as ReplayLog material.
5. A targeted capability module is distilled.
6. The new module is registered with a lineage chain.
7. Future similar tasks are handled locally with lower dependency on the teacher model.

This mechanism is analogous to a semantic version of memory caching or demand paging. A system observes repeated runtime demand, materializes the missing capability, and reduces future fallback cost.

5 Runtime Execution Model

DynamicSLM applies operating-system concepts such as process management, dynamic linking, memory paging, and heterogeneous scheduling to language model execution. AIKernel's runtime does not simply load a model and call it. It resolves an intent, verifies capability contracts, selects a capability subgraph, loads the corresponding payloads, and schedules execution across available compute resources.

The **IPipelineOrchestrator** uses the Execution Profile to place capability modules on CPU, GPU, NPU, or other accelerators. The **IModelVectorRouter** maps task intent to candidate DynamicSLM modules. The **Registry & Loader** verifies lineage, checks compatibility, and materializes only the required payloads.

5.1 Model Hot-Swapping

Model hot-swapping is central to DynamicSLM. In a conventional runtime, loading and unloading a large model can take seconds and consume all available VRAM. DynamicSLM reduces the unit of loading from the whole model to a capability payload. The runtime can page in a required capability and page out an inactive one.

In memory-constrained environments, this enables multiple logical model abilities to coexist over time on a single physical accelerator. The runtime may keep frequently used capabilities resident, prefetch predicted capabilities based on the pipeline graph, and offload lower-priority modules to CPU or NPU resources.

5.2 Trajectory Integration and Context Isolation

When multiple capability modules participate in a task, their outputs must be integrated without allowing uncontrolled interference. AIKernel performs this integration through the Semantic Storage Model. The Semantic Storage Model acts as a deterministic intermediate representation layer: it converts transient module outputs into stable semantic artifacts that can be addressed, replayed, hashed, and passed forward without exposing mutable internal model state. Intermediate artifacts are stored as ReplayLog entries and provided to subsequent modules as immutable context.

This isolates module execution. One capability does not directly mutate another capability's internal state. Instead, capabilities communicate through governed semantic artifacts. This keeps the execution path auditable and reduces nondeterministic cross-module interference.

5.3 Capacity Budgets and Scheduling

DynamicSLM scheduling is not simple FIFO execution. The scheduler considers dynamic capacity budgets, latency service-level objectives, resource pressure, and task priority. An urgent interactive task may receive immediate VRAM allocation, while background differential distillation can be moved to idle CPU or NPU capacity.

The **IComputeShapeAdvisor** monitors hardware load and advises placement decisions. The scheduler can therefore treat models as governed runtime processes rather than static blobs.

6 Conceptual Evaluation

This section is explicitly non-empirical. DynamicSLM is an architectural paradigm and the complete empirical implementation is ongoing. The following evaluation is based on architectural analysis, hypothetical module sizes, expected hardware behavior, and architecturally plausible runtime effects. It should be read as a conceptual assessment rather than as measured benchmark evidence.

6.1 Memory Footprint Expectations

If a typical capability module is approximately equivalent to a 1.0B to 1.5B parameter model segment, DynamicSLM can load only the modules required for the active task. Compared with a monolithic 7B-class SLM, this architecture is expected to reduce peak VRAM usage substantially in workloads that activate only a small portion of the Capability Graph. Depending on the number of active capability nodes, a 50% to 75% reduction in peak memory footprint appears architecturally plausible as a hypothesis, not as an empirical result.

6.2 Expected Swap Latency

Assuming PCIe Gen4 or Gen5 bandwidth and graph-aware asynchronous prefetching, module page-in and page-out latency may fall within the range of tens of milliseconds for appropriately sized capability payloads. This expectation is hypothetical and depends on payload size, quantization format, host-device transfer behavior, memory pinning, and runtime prefetch accuracy. Such latency may be acceptable for interactive agent pipelines such as structure-generation-polish workflows if prefetching is effective and payloads remain small.

6.3 Differential Distillation Efficiency

Replay-based distillation suggests that focused adaptation can improve narrow capabilities with fewer samples than broad retraining. DynamicSLM applies this principle at the capability-node level. Because the distillation target is narrow, improvement may be more data-efficient than retraining a general-purpose model. This remains an architectural hypothesis to be tested empirically.

6.4 Personalization Behavior

DynamicSLM physically isolates user-specific or domain-specific adaptations as capability modules. This suggests a memory-efficient path to multi-user adaptation: users need not each load a full model copy if their adaptations can be represented as separate capability payloads over a shared base. AIKernel can schedule and isolate these payloads according to policy. This is a conceptual claim about deployment structure rather than a measured multi-user benchmark.

7 Discussion

DynamicSLM provides benefits beyond inference efficiency. Because capabilities are explicit runtime objects, AIKernel can govern them before they are loaded. A Policy Decision Point can reject a dangerous capability, such as restricted system command generation, before any token is produced. This is stronger than post-hoc prompt filtering because it prevents the prohibited capability from being materialized in the active runtime boundary.

The architecture also improves auditability. Each capability payload has lineage, and each execution can be recorded through ReplayLog. If a task fails or a capability produces unsafe output, the system can identify which capability was active, which parent model produced it, which distillation traces contributed to it, and which runtime context invoked it.

However, capability modularization introduces trade-offs. If capabilities are too fine-grained, context fragmentation may occur. The system then pays orchestration overhead to bridge semantic gaps among modules. Some tasks may require a coherent global reasoning state that is better served by a larger monolithic model. DynamicSLM should therefore be understood as a runtime architecture for capability-modular workloads, not as a universal replacement for all model deployment styles.

A key future direction is self-organization of the Capability Graph. In the current design, the target node for differential distillation is selected by heuristics or governance policy. In future versions, AIKernel's semantic compiler may analyze ReplayLogs, detect recurring capability gaps, propose new Capability Graph nodes, and generate corresponding distillation processes automatically.

8 Conclusion

This paper proposed **DynamicSLM**, a capability-modular and self-improving small language model architecture for AIKernel. The design reinterprets language models as dynamic processes governed by an operating-system-like runtime rather than as static monolithic binaries.

Through the AIKernel Model ABI, each model artifact exposes a Semantic Profile, Capability Graph, Execution Profile, Lineage, and Payload. AIKernel can therefore decide which capabilities to load, where to schedule them, how to verify their provenance, and how to record their execution. Through differential distillation, the system can progressively reduce dependence on large teacher models by converting verified runtime traces into targeted local capability modules.

DynamicSLM is intended as a step toward Semantic Context Operating Systems in which models are not merely probabilistic generators, but governed, traceable, hot-swappable, and self-improving components of a deterministic execution substrate.

9 References

[1] Sogawa, T. (2026). *AIKernel Trajectory Governance Model: A Kernel-Level Framework for Convergent Decision Control over Stochastic Language Model Inference*. Zenodo.
DOI: https://doi.org/10.5281/zenodo.20309510

[2] Sogawa, T. (2026). *AIKernel Phase-2 Theory: Semantic Compilation Architecture*. Zenodo.
DOI: https://doi.org/10.5281/zenodo.20312092

[3] Sogawa, T. (2026). *AIKernel Semantic DSL Compiler and Deterministic Agent Execution Architecture*. Zenodo.
DOI: https://doi.org/10.5281/zenodo.20534341

[4] Sogawa, T. (2026). *AIKernel Hash-Anchored Trust Layer (HATL): A Hybrid Symmetric Ledger with Hash-Based Public Anchors*. Zenodo.
DOI: https://doi.org/10.5281/zenodo.20502685